Section A: Concept Application

Question 1: Explain how relational databases help maintain accuracy in expense records.

Answer: Relational databases maintain accuracy through **ACID properties** (Atomicity, Consistency, Isolation, Durability) and normalized table structures. Instead of repeating user names or category names across millions of rows—which risks typos and inconsistent data—the database stores data once in dedicated tables (users, categories) and links them using numeric keys. This eliminates data redundancy and guarantees that updates to a user's details instantly reflect across all their related financial records.

Question 2: Why are constraints important in personal finance data?

Answer:

Constraints serve as the fundamental data-quality guardrails for personal finance data:

- **PRIMARY KEY:** Prevents duplicate transactions by ensuring every single expense or user has a unique identifier.
- **FOREIGN KEY:** Prevents data fragmentation by ensuring an expense can never be assigned to a non-existent user or an undefined category.
- **NOT NULL / CHECK:** Prevents broken calculations by ensuring financial values like amount aren't left blank or registered as negative numbers where invalid.

Question 3: How does GROUP BY help analyze spending patterns?

Answer: The GROUP BY clause collapses individual transaction rows into distinct categorical buckets. When paired with aggregate functions like SUM(amount), AVG(amount), or COUNT(), it turns a chaotic list of daily receipts into actionable financial insights (e.g., showing you exactly how much money went to 'Rent' vs. 'Food' over a specific period).

Question 4: Explain a scenario where rollback is required during expense entry.

Answer: Imagine entering a split-bill transaction where money must be deducted from a bank balance table and added to an expenses table. If the system logs the expense row but the application crashes before updating the bank balance, the financial data becomes desynchronized. A ROLLBACK is triggered here to completely undo the partial entries, returning the database to its last clean state as if the broken transaction never happened.

Question 5: How do views help users track monthly expenses efficiently?

Answer: Views act as saved, pre-written queries that mask complex multi-table joins. Instead of typing out intricate INNER JOIN statements involving three tables every time a user checks their monthly dashboard, a view lets the application retrieve real-time aggregated data using a clean, simple SELECT * FROM ViewName statement.

Question 6: Why use triggers for automatic category or balance updates?

Answer:

Triggers eliminate manual calculations by running automatically behind the scenes in response to specified database events (INSERT, UPDATE, DELETE). For instance, whenever a new row is appended to the expenses table, an AFTER INSERT trigger can instantly recalculate and update a current_balance column in a separate user profile table without requiring extra code from the frontend application.